

Exercise1

Sample solution -DFS- C++

```
/* DFS + Enum*/
```

```
#include<iostream>
using namespace std;
```

```
bool chess[6][6]={false};
bool flag;
int step;
int r[]={-1,1,0,0,0};
int c[]={0,0,-1,1,0};
```

```
bool judge_all(void)
{
    int i,j;
    for(i=1;i<5;i++)
        for(j=1;j<5;j++)
            if(chess[i][j]!=chess[1][1])
                return false;
    return true;
}
```

```
void flip(int row,int col)
{
    int i;
    for(i=0;i<5;i++)
        chess[row+r[i]][col+c[i]]=!chess[row+r[i]][col+c[i]];
    return;
}
```

```
void dfs(int row,int col,int deep)
{
    if(deep==step)
    {
        flag=judge_all();
        return;
    }

    if(flag||row==5)return;

    flip(row,col);
    if(col<4)
        dfs(row,col+1,deep+1);
    else
        dfs(row+1,1,deep+1);

    flip(row,col);
    if(col<4)
        dfs(row,col+1,deep);
    else
        dfs(row+1,1,deep);

    return;
}
```

```

int main(void)
{
    char temp;
    int i,j;
    for(i=1;i<5;i++)
        for(j=1;j<5;j++)
        {
            cin>>temp;
            if(temp=='b')
                chess[i][j]=true;
        }

    for(step=0;step<=16;step++) //Enumerate the possibilities generated at each step
    {                             //As for why it is 16, considering that 4x4=16 grids,
                                //and each grid only has black and white cases,
                                //then the total possibility is 2^16

        dfs(1,1,0);
        if(flag)break;
    }

    if(flag)
        cout<<step<<endl;
    else
        cout<<"Impossible"<<endl;
    return 0;
}

```

Equivalent in python-DFS

```
# The board is a 6x6 array, but we only use the central 4x4 area (indices 1-4)
chess = [[False] * 6 for _ in range(6)]

# Variables similar to C++ version
flag = False
step = 0

# Directions for flipping (up, down, left, right, center)
r = [-1, 1, 0, 0, 0]
c = [0, 0, -1, 1, 0]

def judge_all():
    """Check if all pieces are the same color."""
    first = chess[1][1]
    for i in range(1, 5):
        for j in range(1, 5):
            if chess[i][j] != first:
                return False
    return True

def flip(row, col):
    """Flip the current piece and its four neighbors."""
    for i in range(5):
        nr, nc = row + r[i], col + c[i]
        chess[nr][nc] = not chess[nr][nc]
```

```
def dfs(row, col, deep):  
    """Depth-first search with backtracking."""  
    global flag  
  
    # If we have reached the current step limit, check board status  
    if deep == step:  
        flag = judge_all()  
        return  
  
    # If already solved or out of bounds, stop  
    if flag or row == 5:  
        return  
  
    # 1. Flip current position and move forward  
    flip(row, col)  
    if col < 4:  
        dfs(row, col + 1, deep + 1)  
    else:  
        dfs(row + 1, 1, deep + 1)  
    flip(row, col) # Backtrack (undo the flip)  
  
    # 2. Skip flipping current position and move forward  
    if col < 4:  
        dfs(row, col + 1, deep)  
    else:  
        dfs(row + 1, 1, deep)
```

```

def main():
    global flag, step

    # Read input
    for i in range(1, 5):
        line = input().strip()
        for j in range(1, 5):
            if line[j - 1] == 'b':
                chess[i][j] = True

    # Enumerate possible number of steps from 0 to 16
    for step in range(17): # 0 to 16 inclusive
        dfs(1, 1, 0)
        if flag:
            break

    # Output result
    if flag:
        print(step)
    else:
        print("Impossible")

if __name__ == "__main__":
    main()

```

Sample solution -bitmask- python

```

from collections import deque

# Precompute flip masks for each position on 4x4 board
def build_flip_masks():
    masks = []
    for i in range(4):
        for j in range(4):
            mask = 0
            # Center
            mask |= 1 << (i * 4 + j)
            # Four neighbors
            if i > 0: mask |= 1 << ((i - 1) * 4 + j)
            if i < 3: mask |= 1 << ((i + 1) * 4 + j)
            if j > 0: mask |= 1 << (i * 4 + (j - 1))
            if j < 3: mask |= 1 << (i * 4 + (j + 1))
            masks.append(mask)
    return masks

def parse_input():
    """Read input and return initial board state as a 16-bit integer."""
    state = 0
    for i in range(4):
        line = input().strip()
        for j, ch in enumerate(line):
            if ch == 'b':
                state |= 1 << (i * 4 + j)
    return state

```

```

def bfs_min_steps(start, masks):
    """BFS to find minimum flips to make board uniform."""
    target_states = {0, (1 << 16) - 1} # all white or all black
    if start in target_states:
        return 0

    visited = {start}
    queue = deque([(start, 0)])

    while queue:
        state, steps = queue.popleft()

        for mask in masks:
            next_state = state ^ mask # flip operation
            if next_state in target_states:
                return steps + 1
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, steps + 1))

    return -1 # Impossible

def main():
    masks = build_flip_masks()
    start = parse_input()
    result = bfs_min_steps(start, masks)
    print(result if result != -1 else "Impossible")

if __name__ == "__main__":
    main()

```

Sample solution -bitmask- C++

```

#include <stdio>
#include <queue>
#include <algorithm>

int dp[1<<16+10] = {0};
const unsigned int Full_Black = (1<<16)-1;
const unsigned int Full_White = 0;

int bfs(unsigned int bit);
unsigned int toggle(unsigned int bit, int pos);

int main()
{
    unsigned int bit = 0;
    char line[5];
    for (int i = 0; i < 4; ++i) {
        scanf("%s", line);
        for (int j = 0; j < 4; ++j) {
            bit <<= 1;
            if (line[j] == 'b') bit |= 1;
        }
    }
    if (bit == Full_Black || bit == Full_White) puts("0");
    else {
        int ans = bfs(bit);
        if (ans == -1) puts("Impossible");
        else printf("%d\n", ans);
    }
}

```

```

int bfs(unsigned int start_bit)
{
    std::queue<unsigned int> Q;
    Q.push(start_bit);

    while (!Q.empty()) {
        unsigned int bit = Q.front();
        Q.pop();
        for (int i = 0; i < 16; ++i) {
            unsigned int toggled_bit = toggle(bit, i);
            if (!dp[toggled_bit]) {
                dp[toggled_bit] = dp[bit] + 1;
                if (toggled_bit == Full_Black || toggled_bit == Full_White)
                    return dp[toggled_bit];
                Q.push(toggled_bit);
            }
        }
    }
    return -1;
}

unsigned int toggle(unsigned int bit, int pos)
{
    bit ^= (1 << pos); // central
    if (pos-4 >= 0) bit ^= (1 << (pos-4)); // up
    if (pos+4 < 16) bit ^= (1 << (pos+4)); // down
    if (pos%4 >= 1) bit ^= (1 << (pos-1)); // left
    if (pos%4 <= 2) bit ^= (1 << (pos+1)); // right
    return bit;
}

```

Bitmask

- a.k.a. lightweight set of Boolean or bit string

- Explanation via:

<https://visualgo.net/en/bitmask>

Representing Set

- We have a small ($n \leq 30$) number of items
- We label them with integers in the range $0, 1, \dots, n-1$
- We can represent sets of these items as a 32-bit integer
- The i th item is in the set represented by the integer x if the i th bit in x is 1
- Example:
 - We have the set $\{0, 3, 4\}$
 - `int x = (1<<0) | (1<<3) | (1<<4);`

Representing Set

- Empty set:
 0
- Single element set:
 $1 \ll i$
- The universe set (i.e. all elements):
 $(1 \ll n) - 1$
- Union of sets:
 $x | y$
- Intersection of sets:
 $x \& y$
- Complement of a set:
 $\sim x \ \& \ ((1 \ll n) - 1)$

Representing Set

Check if an element is in the set:

```
if (x & (1<<i)) {
```

```
    // yes
```

```
} else {
```

```
    // no
```

```
}
```

Representing Set

- Why do this instead of using `set<int>`?
 - Very lightweight representation
- All subsets of the n elements can be represented by integers in the range $0 \dots 2^n - 1$
- Allows for easily iterating through all subsets (we'll see this later)
- Allows for easily using a set as an index of an array (we'll see this later)

Applications of Sets

- Keep track of distinct items
- Have we seen an item before?
- If implemented as a binary search tree:
 - Find the successor of an element (the smallest element that is greater than the given element)
 - Count how many elements are less than a given element
 - Count how many elements are between two given elements
 - Find the k th largest element
- And a lot more